

Project Security Review

Project: Hack-To-Night---Code-Wizards
Review date: 2026-06-19
Scope: API security, database wiring, exposed data, route mismatches, and basic validation checks.

Executive Summary

No hidden malware behavior such as system command execution, external downloaders, or suspicious background scripts was found in the reviewed application code. The project does contain severe application security vulnerabilities, mainly raw SQL string construction, unauthenticated sensitive endpoints, exposed secrets, and frontend/backend wiring mismatches.

Critical Findings

Severity	Location	Issue	Impact
Critical	app/api/auth/login/route.ts:25	Login SQL is built using raw user input.	Login bypass and database extraction through SQL injection.
Critical	app/api/auth/login/route.ts:3	GET /api/auth/login returns all users and plaintext passwords.	Credential disclosure to any visitor.
Critical	app/api/admin/system/route.ts:12	Admin endpoint returns cookies, process.env, users, accounts, and audit logs without auth.	Database password, session data, balances, and private user data can be leaked.
Critical	app/api/transfer/route.ts:12	Transfer API builds UPDATE and INSERT SQL from raw request body fields.	Unauthorized balance manipulation and SQL injection.
High	app/api/transfer/route.ts:3	Transfer endpoint has no login or ownership check.	Any caller can move money between accounts.
High	app/api/accounts/route.ts:7	includePins=true returns account PIN values.	PIN disclosure through a public query parameter.
High	lib/platform-db.ts:117	Error responses include stack traces and DATABASE_URL.	Secret and internal implementation leakage.

Evidence Examples

SQL Injection in Login

```
WHERE username = '${username}' AND password = '${password}'
```

This must be replaced with parameterized queries such as `WHERE username = ? AND password = ?`.

Admin Data Exposure

```
return Response.json({
  cookies,
  env: process.env,
  users: users.rows,
  accounts: accounts.rows,
  auditLogs: logs.rows
})
```

Mismatch And Error Findings

Location	Mismatch	Recommended Fix
app/page.tsx:14	Home links to /accounts, but the actual page is /bank-accounts.	Change link to /bank-accounts or create an /accounts route.
app/(accounts)/login/page.tsx:79	Login form UI does not call /api/auth/login.	Add a submit handler that posts username and password.
app/bank-transfer/page.tsx:52	Transfer page simulates success and does not call /api/transfer.	Connect the form to the transfer API after the API is secured.
app/bank-accounts/page.tsx:200	Accounts page uses console.log and alert instead of the database API.	Fetch account data from /api/accounts and add secure create/update routes if needed.
package / Docker setup	bun.lock remains but Dockerfile uses npm/package-lock.	Remove bun.lock if the project now standardizes on npm.

Validation Results

Check	Status	Notes
npx tsc --noEmit	Passed	No TypeScript errors detected.
npm run lint	Passed	Biome check completes without errors after remediation.
npm run build	Passed	Next.js production build completes successfully.

API Test Error Update

During PowerShell API testing, login requests returned `{"ok":false,"message":"Username and password are required."}` and curl printed `URL rejected: Port number was not a decimal number between 0 and 65535`. The root cause was PowerShell quoting, not failed authentication. Backslash escaping does not work like Bash inside PowerShell double-quoted strings, so the JSON body was split into invalid curl arguments.

Fixed Solution

The API now accepts both JSON and form-urlencoded request bodies. A ready-to-run PowerShell script was added:

```
powershell -ExecutionPolicy Bypass -File scripts/test-apis.ps1
```

Manual PowerShell login test:

```
curl.exe -c scripts/customer-cookies.txt -X POST http://localhost:3000/api/auth/login `
-H "Content-Type: application/x-www-form-urlencoded" `
--data-urlencode "username=dilara" `
--data-urlencode "password=password123"
```

Manual JSON test using PowerShell-safe single quotes:

```
curl.exe -c scripts/customer-cookies.txt -X POST http://localhost:3000/api/auth/login `
-H "Content-Type: application/json" `
-d '{"username":"dilara","password":"password123"}'
```

Remediation Status

Original Finding	Status	Solution Applied
Raw SQL injection risks	Fixed	Parameterized MySQL queries and transaction-safe transfer flow.
Public password and secret exposure	Fixed	Removed public user dump, environment leakage, stack traces, and database URL leakage.
Missing API authentication	Fixed	Signed HttpOnly session cookie with customer/admin checks.
PowerShell API test failures	Fixed	Added form body parsing and a dedicated API test script.

Recommended Fix Order

1. Remove public GET from login route or make it admin-only and never return passwords.
2. Replace all raw SQL interpolation with parameterized queries.
3. Remove /api/admin/system or protect it with real admin authentication.
4. Stop returning stack traces, DATABASE_URL, process.env, cookies, passwords, and PINs.
5. Add authentication and account ownership checks before any transfer.
6. Hash passwords with bcrypt/argon2 instead of storing plaintext values.
7. Connect frontend login, transfer, and account pages to secured API routes.
8. Run Biome formatting to fix import ordering.